THE UNIVERSITY *of* EDINBURGH
School of Physics
and Astronomy

Scientific Image Analysis

Automatic Recognition of COVID-19 Lateral Flow Test Results

P. Antonopoulos
March 2024

Executive Summary

In this project, an image analysis pipeline to automatically recognise COVID-19 Lateral Flow Test results was made by developing algorithms to automatically pre-process user supplied images and to detect lines corresponding to the control and test lines. On the 13 supplied images plus an extra image to test void results, the algorithms rejected 4 images due to low contrast and achieved a 100% accuracy on the remaining 10 in a total runtime of 38 seconds. To minimise risk of image rejection due to low contrast, supplied images would preferably be evenly illuminated with a dark background. Additionally, further studies have been proposed to make the algorithms more robust to images of differently oriented LFT devices.

Declaration

I declare that this project and report is my own work.

Contents

1 Introduction

1.1 Motivation

During the COVID-19 pandemic, in an effort to maintain control on the spread of the virus from tourists, governments implemented travel restrictions which required proof of a negative COVID-19 lateral flow test (LFT) result to be allowed entry into their country [1]. In order for such a system to be effective, visitors were required to obtain a certificate that demonstrated a ”fit-to-fly” status which was granted upon independent verification of the LFT result.

This requirement was subsequently commercialised with the emergence of companies offering services to verify LFT results. Such a process involved the customer being sent a LFT to complete together with instructions on how to take a photo of the result [1]. Once this was done, the customer would upload the image to a website which would review the photo and email back a confirmation of the test result with an accompanying certificate [1].

To make such a procedure sustainable, it was necessary to develop an image analysis pipeline to automatically recognise LFT results from such photos.

Apart from this algorithm being used for the purpose of independently verifying LFT results for international travel, the automatic recognition of LFT results is key to preparing for future pandemics which are increasingly being seen as inevitable [2]. This is because automatically recognising LFT results on a national scale will facilitate rapid processing of data which will aid monitoring the spread of disease through the population.

1.2 Background Theory

The two most common tests used during the COVID-19 pandemic to ascertain whether a person was infected were the polymerase chain reaction (PCR) test and the LFT. The PCR test involves a swab of the nose or throat being used to search for viral genetic material by amplifying DNA [3]. This method is extremely accurate, however due to requiring laboratory processing [3] it is constrained by lab capacity on top of being expensive and slow to return results [4].

The LFT - also known as a rapid antigen test - was widely used by the public as a popular alternative to PCR tests due to it circumventing a number of their disadvantages, such as by providing results within 30 minutes [5]. LFTs are not unique to COVID-19, perhaps the most well known example of a LFT is a pregnancy test, from which the principle has been expanded to develop LFTs to identify diseases such as malaria and HIV [5]. A COVID-19 LFT works by inserting a swab of the nose or throat into a buffer solution which is then dripped into the sample hole (marked ”S”) of the test strip. Once the solution enters the strip, antibodies are released which capture antigens of the virus if they are present [6], the solution then begins to travel up the strip by capillary action [5]. In the case that antigens were present, the sample will bind to subsequent immobilised antibodies located in the test area (marked ”T”) which causes a change in colour - producing the test line which represents a positive result [6]. On the other hand, in the absence of antigens, the solution will pass the test area and continue to move up to the control area (marked ”C”) where the antibodies will bind to other immobilised antibodies,

creating another change in colour which forms the control line [6].

The control line indicates the validity of the test. The presence of a control and test line shows a positive result, the presence of just the control line marks a negative result while a void result is characterised by the absence of the control line regardless of the status of the test line.

1.3 Approach to the Problem

In the project pitch, the general method that was proposed was based around pixel intensities. Given an inverted grayscale image that was cropped to show only the test strip region (the region of interest - ROI), any present control and/or test lines would be formed of whiter pixels (with larger pixel values) while the background would be darker (with lower pixel values). Two approaches to follow this method were proposed, the first was to take line profiles along the length of the test strip and find peaks in the pixel values above a threshold to ascertain if a line was present, this approach is illustrated below using a ROI obtained from manual pre-processing to test the logic.

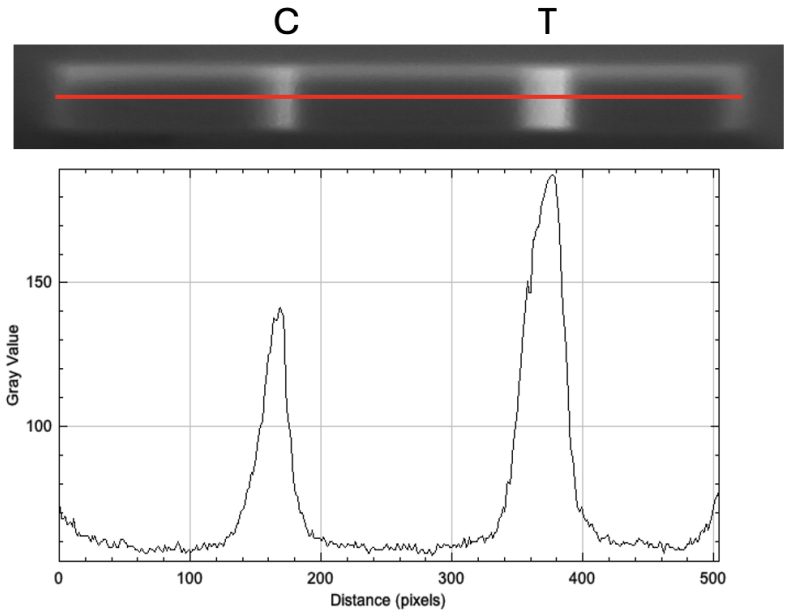

Figure 1: Illustration of the line profile approach on a positive test strip for the horizontal red line. Two clear peaks in the pixel value can be seen which correspond to the control and test lines. Manual pre-processed ROI and graph obtained using the ImageJ image analysis software.

In this approach, 2000 random line profiles were taken instead of the initial 5 that was proposed so that the statistics were improved especially in images where the background was not as dark as in Figure 1. Additionally, instead of requiring that all line profiles indicate the presence of a line, confirmation of a control line was made if 90% of line profiles indicated its presence and similarly a test line accept condition of 95% of line profiles was used. Finally, the requirement that the detected lines across each line profile must be at similar positions was dropped to simplify the algorithm for code readability.

The second approach iteratively took slices along the width of the ROI and computed the average pixel value within each slice to determine whether that slice was part of a line. The first modification to this approach was that instead of detecting a potential line if the increase in average pixel value from the previous slice was above a particular threshold, a potential line was identified if the average pixel value of a slice itself was above a threshold. This change was made to simplify the code logic and hence improve the readability without compromising accuracy. The second modification was to exclude the top and bottom 5% of the ROI when taking slices so that edge effects caused by shadows were not taken into account. This approach will be further detailed in the Methods section.

In this project, both of the aforementioned approaches were implemented in Python and were tested against the supplied images which had been manually pre-processed. It was found that the second approach had a higher accuracy, a quicker runtime and greater readability. Therefore, the second approach was used in the remainder of the project to analyse the images.

Finally, the timeline of the project as set out in the pitch included time to experiment whether analysing colour images instead of converting them to grayscale would improve the accuracy of the algorithm. It was found that once the ROI had been established, the algorithm detected all test strip lines successfully, therefore no improvement could be made to result determination for the images that were supplied for the project. However, it was proposed that using colour images would be able to aid the identification of the ROI as fewer images may be classified as having too low contrast. Unfortunately, when colour images were attempted to be used in the automatic data pre-processing procedure, it was noted that the Python library used was not able to effectively handle colour images as the runtime increased to unfeasible lengths. Therefore, the supplied images remained to be converted to grayscale for analysis.

1.4 Report Structure

This report presents a comprehensive overview of the work undertaken in this project, including the methods to develop the algorithm (§2) and the results it produces for the test images provided in the brief (§3). A critical discussion of the results and methods is then made (§4) before concluding the findings of the project (§5).

2 Methods

The methods that were employed for this project consisted of automating the image pre-processing procedure and developing the algorithm to automatically detect the test strip line(s). This was done using the Python programming language in a Jupyter Notebook.

2.1 Image Pre-Processing Algorithm

As alluded to in §1.3, the algorithm to detect line(s) required the input to be the test strip that had been converted to grayscale and inverted. This is illustrated in the figure below.

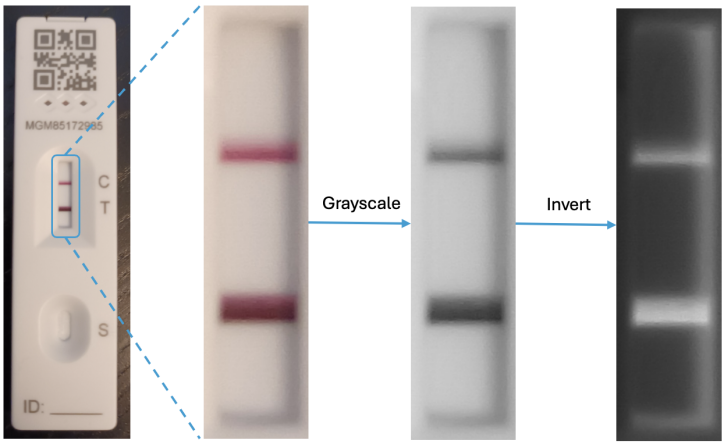

Figure 2: Illustration of the image pre-processing procedure.

To implement an algorithm that did this automatically, image segmentation methods were used to initially locate the LFT device. To do this, the colour image was first converted to grayscale and by experimenting with the outputs, it was decided to perform an image enhancement operation to enhance the contrast. This was done using the `skimage.equalize_adapthist` function which employs an algorithm for local contrast enhancement using the technique of contrast limited adaptive histogram equalisation. Following this, the algorithm determined whether the image was landscape or portrait by computing the number of rows and columns of the image. If the orientation was landscape, the image was rotated 90^{o} clockwise so that the test strip was upright as shown in Figure 2. Then, the image was binarised through using the `skimage` otsu threshold function and using the result, the algorithm employed the `skimage .measure.label` function to label connected regions in the image by assigning such regions with the same integer value. This returned a labelled numpy array that was subsequently used in the `skimage.measure.regionprops` function which measured various properties of each labelled region in the image and returned a list holding all the region properties.

To proceed with the image segmentation, the algorithm iterated through each region and stored two attributes that were measured, such as the bounding box coordinates (where the bounding box is a box which encloses the entire region) and the area of the region. At this point, it was determined which region was most likely to be the the LFT device by selecting the object with the largest area. This was done because it was unlikely that the image would contain a physical object larger than the LFT device. Following this, the algorithm determined whether the image was not able to be analysed due to low contrast. It did this by using the bounding box coordinates to determine the ratio between its width and height. Using physical dimensions to obtain a known aspect ratio of the LFT device and accounting for slight variations due to imperfect region identification, if the bounding box W:H ratio did not fall into an acceptable range then it was determined that the bounding box did not include the whole LFT device or the LFT device on its own without any surrounding regions. In such a case, the LFT device and hence the test strip was deemed as unrecognisable and the image was rejected by exiting the function and printing a statement to the user.

For the images that were deemed acceptable quality, the ROI was located by computing the centre coordinate of the bounding box and using physical dimensions of the LFT device to

determine characteristic ratios of the distance from the bounding box centre to the edges of the LFT device with the distance from the centre to the edges of the ROI. These ratios were then used to form upper and lower intervals from the centre of the LFT device which were used to crop the image to leave just the ROI. Lastly, the ROI was then inverted and returned to be used in the algorithm to detect lines.

The length ratios used in this algorithm are detailed in Appendix A. Additionally, the automatic pre-processing algorithm can be nicely summarised by the following flowchart.

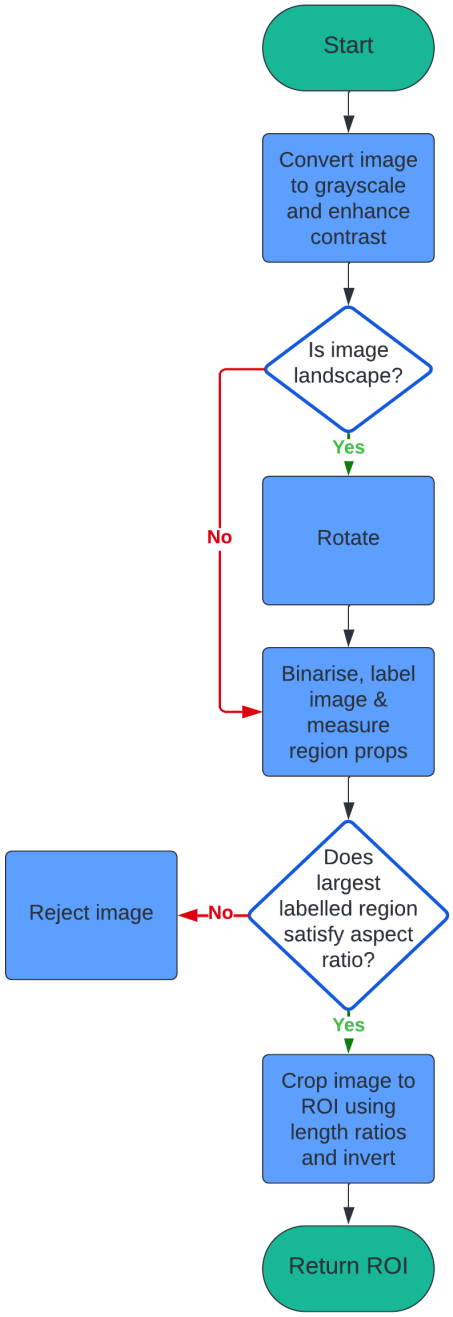

Figure 3: Flowchart for the automated line detection algorithm. Green ovals represent terminators, diamonds represent decisions and blue rectangles represent actions.

2.2 Line Detection Algorithm

The approach used to detect lines in the ROI made use of the fact that they would be whiter and hence have larger pixel values than the background which would be darker with corresponding lower pixel values. This approach was to develop an algorithm where it will iteratively take successive slices along the width of the test strip and compute the average pixel value in each slice, this is illustrated in the below figure.

Figure 4: Illustration of the approach used in the project to detect the test strip line(s). The ROI is displayed landscape for illustration purposes.

In this approach, it can be seen that the slices outlined in blue will have a low average pixel value due to them containing darker background pixels. However, the slices outlined in red will have a higher average pixel value due to the presence of a line. During the process of making successive slices along the ROI, if the average pixel value of a slice exceeded an amplitude threshold value, then the algorithm would detect a potential line. To confirm or reject the potential line, the algorithm would monitor how long such high average values were sustained to measure the thickness of the line and if the thickness exceeded a thickness threshold, then the presence of a line would be confirmed in the image. This was to make the algorithm robust to noise so that rows forming the background which happened to have a average value above the threshold were not mistakenly identified as a line.

To implement this algorithm, the pixel value threshold was set as the average pixel value of the whole image. This was to account for variations in the brightness of the background in the pre-processed ROI as regardless of the background brightness, the line(s) would be brighter and hence the average pixel value of the rows making up the line(s) would be above the average value of the whole image. Following this, the algorithm determined the number of rows of the ROI and initialised the status of the control and test line to `False` (i.e. undetected).

At this point, it began to loop over all the rows of the ROI except from the first and last 5% of rows, this was to avoid edge effects at the top and bottom of the ROI due to potential shadows which would appear bright when inverted. To take a slice, the algorithm indexed the image to extract a row and then computed the average pixel value in that slice, it then checked whether the average value was above the threshold value. If so, then a potential line was detected and a counter which monitored how many consecutive rows this was sustained for (i.e. the thickness of the line) was increased by 1. Using physical dimensions of the ROI to determine characteristic ratios of line thickness to the size of the ROI, if the thickness reached 5% of the ROI height, then the presence of a line was confirmed and the algorithm then determined whether it was the control or test line depending on if the line's midpoint was in the upper

(control line) or lower (test line) half of the ROI. If during the process of taking successive slices, the average pixel value of a row did not exceed the threshold, then the counter was reset to 0 as the algorithm would assert that the row did not form part of a line and any potential lines detected directly before that which were not yet confirmed were in fact part of the background. The length ratio is once again presented in Appendix A.

After the algorithm iterated over all the rows, the confirmed lines were used to determine the test result. If both the control and test line were `True` (i.e. confirmed), then the algorithm returned a positive result, if only the control line was `True`, then the algorithm returned a negative result and if the control line was not detected then the algorithm returned a void result. This algorithm can be summarised in the below flowchart.

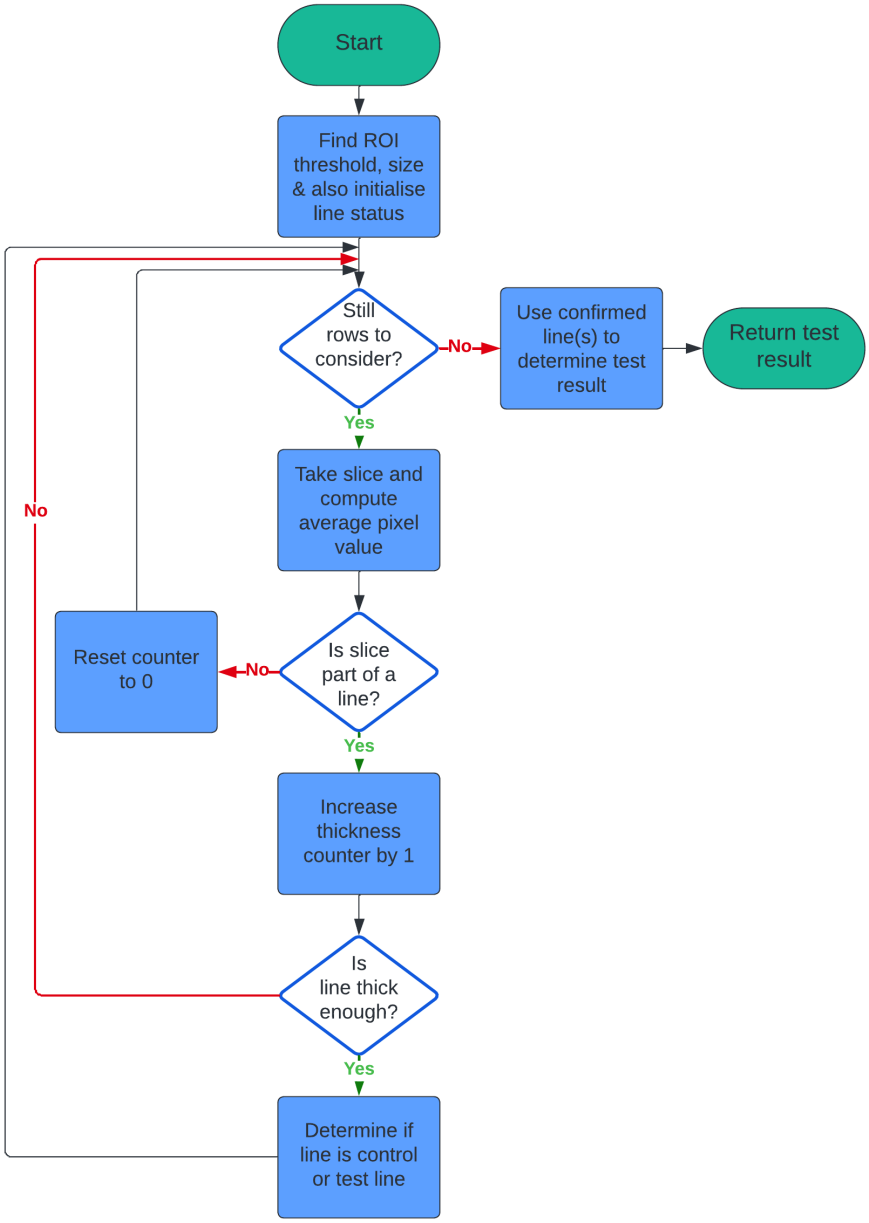

Figure 5: Flowchart for the automated image pre-processing procedure. Green ovals represent terminators, diamonds represent decisions and blue rectangles represent actions.

2.3 Running the Algorithms

To run the algorithms, a directory to the folder storing all the supplied LFT images was specified and the Python `Path` library was used together with the `glob()` module to retrieve all `.jpg` files. Each file was then iterated over and the `skimage.io.imread` function was used to load the image. Once the image was loaded, it was passed to the pre-processing algorithm detailed in (§2.1).which returned the ROI that was then passed to the line detection algorithm detailed in (§2.2) to analyse and return the test result.

Finally, as no void LFTs were provided, a synthetic one (testvoid.jpg) was made by covering the control line of the LFT_00.jpg image so that the algorithm could be tested for this case.

3 Results

3.1 Visualisation of Image Pre-Processing

A visualisation of the automated image pre-processing was obtained and is shown below.

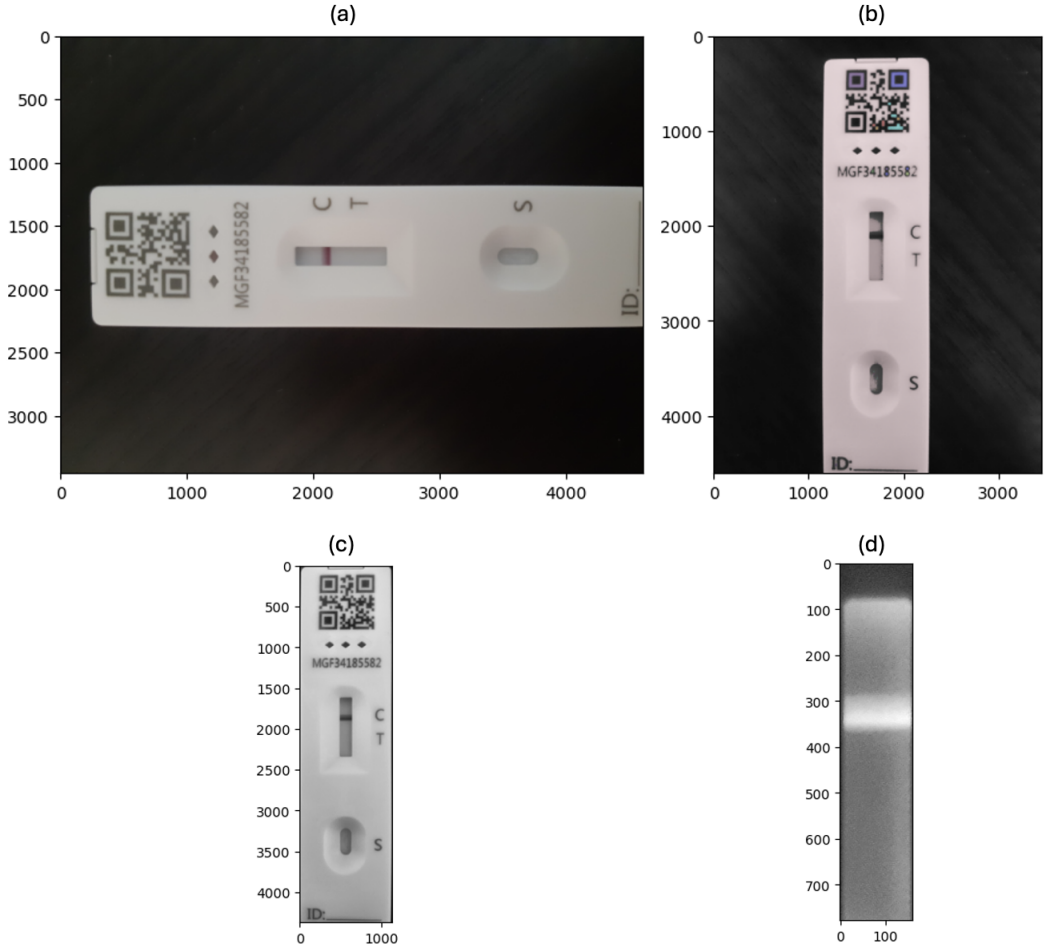

Figure 6: Illustration of the automated pre-processing procedure. (a) is the original image, (b) is the original image that has been rotated with regions labelled and measured, (c) is the identified LFT device (largest labelled region) and (d) is the ROI which has been inverted.

3.2 Final Results

Using the output of the pre-processing algorithm as the input to the line detection algorithm, the following LFT test results were obtained in a total runtime of approximately 38 seconds.

Image	Output	True	Correct?
KFT_04.jpg	N/A	N	-
LDT_14.jpg	P	P	✓
LDR_13.jpg	N	N	✓
LFT_03.jpg	N/A	N	-
LFT_02.jpg	N	N	✓
LGT_05.jpg	N/A	N	-
LFT_00.jpg	P	P	✓
LFT_01.jpg	N	N	✓
LFT_10.jpg	N/A	N	-
testvoid.jpg	V	V	✓
LFT_07.jpg	P	P	✓
LFT_08.jpg	N	N	✓
LFY_06.jpg	N	N	✓
PFT_11.jpg	N	N	✓

Table 1: Results of the algorithms. Images with entries marked "N/A" were rejected due to low contrast. Entries marked "P", "N" and "V" stand for positive, negative and void results, respectively.

4 Discussion

The results presented in Table 1 represent a 100% accuracy for the provided images that were not rejected due to low contrast. Additionally, the code runtime is relatively fast which facilitates rapid result determination.

By considering Figure 6, it has been demonstrated that if required, the algorithm is able to successfully rotate a landscape oriented image, it is then able to correctly label regions as can be seen from the different coloured objects in Figure 6 (b), such as yellow and purple colours for parts of the QR code, and the pale colour of the LFT device. The algorithm then successfully locates the LFT device as illustrated in Figure 6 (c) before inverting the image and correctly utilising the characteristic length ratios to crop the image to the ROI shown in Figure 6 (d). By comparing Figure 6 (d) with the final image shown in Figure 2, it can be seen that the algorithm successfully obtained the ROI in the originally intended format. The only difference in the process was that the image was converted to grayscale first before cropping. This was due to the observation made in §1.3 that the runtime increased to unfeasible lengths when colour images were attempted to be used in the `regionprops` function.

It should be noted that the method to rotate a landscape image (90^{o} clockwise rotation) assumed that the test strip was oriented in the same way as 6 (a). It can be understood that if an

image contained a LFT device that was oriented in the direction opposite to 6 (a) then after a 90^o clockwise rotation the LFT device would be upside down. This would undoubtedly have consequences on the result determination. Because the centre of the LFT device is not at the centre of the test strip region, it is unlikely that the ROI would be identified correctly in the first place due to the approach used. However, supposing that the ROI was identified and then passed to the line detection algorithm, it can be understood that due to the LFT device being upside down, what would be a negative result will be classified as a void result and vice versa due to the expected locations of the control and test line. This is illustrated below.

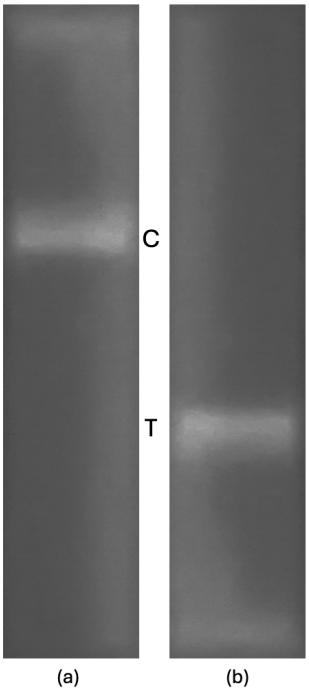

Figure 7: (a) Shows the upright ROI, (b) shows the ROI flipped upside down. ”C” and ”T” show the expected locations (i.e. in the upper and lower half) of the control and test line, respectively. (a) will be correctly classified as negative whereas (b) will be incorrectly classified as void due to the ”absence” of the control line.

This could have potentially serious consequences as incorrect results would then be returned to the customer.

Similarly, it can be appreciated that if the LFT device was oriented in any direction other than upright in a portrait photo then incorrect results would also be returned to the customer. This is because the assumption made in the pre-processing algorithm was that a portrait photo would be of an upright LFT device.

Further work may be done to the automated pre-processing algorithm to make it more robust to images of differently oriented LFT devices. One way to achieve this would be to employ the `orientation` attribute from the `regionprops` function to obtain the angle of the detected LFT device region. The image could then be rotated using this angle so that the LFT device is straight. This was tested by modifying the pre-processing algorithm for a test image, the resulting process is shown below.

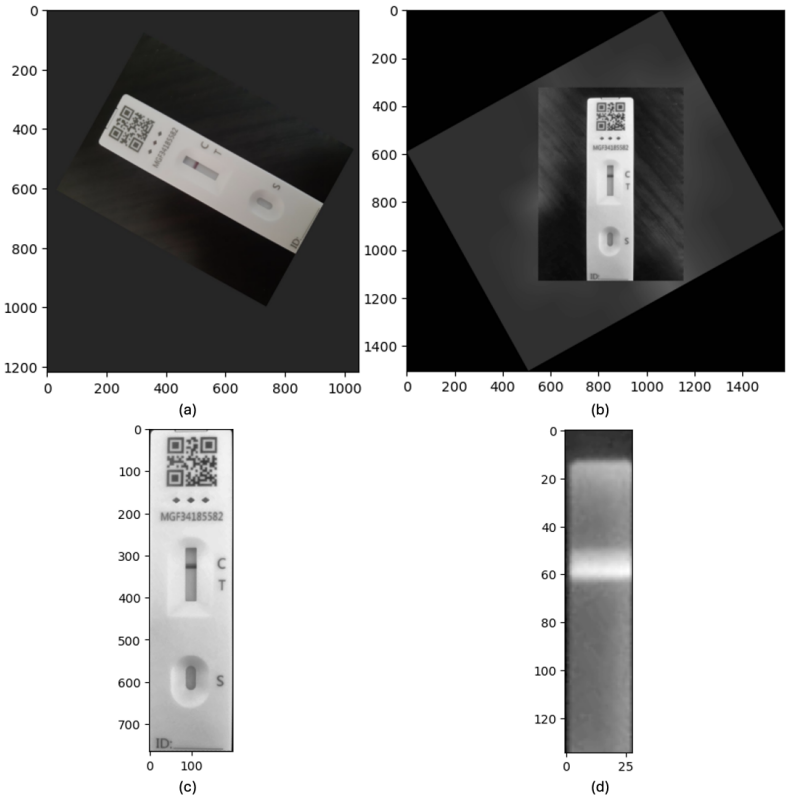

Figure 8: (a) Original image, (b) rotated image, (c) located LFT device, (d) identified ROI.

It can be seen that the procedure was successful, however the risk of the device being upside down is not circumvented yet. To address this, further studies may be done to locate features such as the QR code. Using the fact that in an upright orientation the QR code should be at the top of the device, if it was found to be at the bottom then the image could be rotated 180^{o}.

It should also be noted that the `label` function utilised the assumption that the background pixels are 0 valued, i.e. black. Therefore, to minimise the risk of an image being rejected due to low contrast, the segmentation of the LFT device from the surroundings will work best for an image with a dark background and an evenly illuminated LFT device to avoid shadows on the device that may be classed as background.

5 Conclusions

In conclusion, this report presented a comprehensive overview of the image analysis pipeline that was developed to automatically recognise LFT results. The client brief has been met as all of the essential and desirable criteria have been attained. On the 13 test images supplied plus one to test the void result, the algorithms rejected 4 due to low contrast and on the remaining 10 images achieved a 100% success rate in a total runtime of 38 seconds. Future work has also been proposed to make the image pre-processing procedure more robust by detecting specific features of the LFT device so that an image with the device oriented upside down does not lead to an incorrect result. Finally, it has been noted that to minimise the risk of image rejection due to low contrast, images would preferably be evenly illuminated with a dark background.

References

[1] Cellmark, "Pre-departure lateral flow testing," https://covid19.cellmark.co.uk/services/pre-departure-lateral-flow-testing/, accessed on 28, March, 2024.

[2] W. Dodds and W. Dodds, "Disease now and potential future pandemics," *The world's worst problems*, pp. 31–44, 2019.

[3] G. Guglielmi, "Rapid coronavirus tests: a guide for the perplexed," *Nature*, vol. 590, no. 7845, pp. 202–205, 2021.

[4] UK Government, "Performance of lateral flow devices during the covid-19 pandemic," https://www.gov.uk/government/publications/lateral-flow-device-performance-data/performance-of-lateral-flow-devices-during-the-covid-19-pandemic, April 2023, accessed on 29, March, 2024.

[5] J. Budd, B. S. Miller, N. E. Weckman, D. Cherkaoui, D. Huang, A. T. Decruz, N. Fongwen, G.-R. Han, M. Broto, C. S. Estcourt *et al.*, "Lateral flow test engineering and lessons learned from covid-19," *Nature Reviews Bioengineering*, vol. 1, no. 1, pp. 13–31, 2023.

[6] Una Health, "The rise of the lateral flow test: everything you need to know about lateral flow tests in five steps," https://unahealth.co.uk/blog/the-rise-of-the-lateral-flow-test-everything-you-need-to-know-about-lateral-flow-tests-in-five-steps/, August 2021, accessed on 29, March, 2024.

A LFT Device Length Ratios

Figure 9: Using physical dimensions of the LFT device to determine characteristic length ratios.

Ratio	Value
H/W	$3< x <4.5$
0.5W/a	7
0.5H/l	16
0.5H/b	3.4
H/c	20

Table 2: Ratios obtained using ImageJ that were used in the algorithms.